תרגיל מסכם

ניתן בתאריך: 2 בפברואר 2026

להגשה עד: 8 במרץ 2026

בתרגיל הזה תבצעו הערכה של וריאציה של מודל Whisper עבור תמלול שמע בעברית שפיתחה חברת IvritAI. את המודל תוכלו להוריד מן הקישור הבא, וכן תמצאו שם דוגמה לקוד פשוט לתמלול של קובץ שמע (למעשה הרצה של הקוד מורידה כבר לבד את המודל למחשב שלכם):

https://huggingface.co/ivrit-ai/whisper-large-v3-turbo-ct2

כעת, הורידו את בסיס הנתונים של CommonVoice עבוד השפה העברית מן הקישור הבא:

https://commonvoice.mozilla.org

באתר תוכלו גם לקרוא על הפרויקט של מוזילה לאיסוף שמע, לבחון את השפות הזמינות וכד'. בעיקרון החומר המוקלט מבוסס על הקלטות שביצעו מתנדבים, שנכנסו לאתר והקליטו את עצמם קוראים משפטים שהוצגו להם. הקלטות אלה עברו אחר-כך אימות על-ידי מתנדבים אחרים, ורק קובץ שקיבל שני אימותים (כלומר שנראה שאכן מה שהוקלט אכן תואם את הטקסט) נכנס לחומר האימון או הבדיקה.

כפי שתראו, בסיס הנתונים כולל, פרט לקבצי האודיו המוקלטים עצמם, מספר קבצי tsv שמכילים מידע על ההקלטות השונות, כמו נתונים על זהות המקליט (מיל, גיל וכד') – והחשוב מכל: הטקסט המוקלט.

אנו נתעניין רק בסט הבדיקה: הקובץ `test.tsv` שיהיה הבנצ'מארק שלנו.

שימו לב: כיוון ששדה הטקסט מכיל לעיתים גרשיים או תווים "בעייתיים" אחרים, חלק מן הספריות הסטנדרטיות (כגון csv) אינן קוראות את הקובץ כפי שצריך. לכן מומלץ לקרוא את הקובץ שורה אחרי שורה, ולחלק כל שורה לשדות (מלבד שורת הכותרת כמובן) בעצמכם ע"י: `line.split('\t')`

חלק א' – ביצוע התמלול הבסיסי

בשלב ראשון, בצעו תמלול של הבנצ'מארק. השלב הזה עלול לקחת כמה שעות של ריצה, ולכן מומלץ לבצע אותו פעם אחת ולשמור את התוצאות לקובץ תוצאות. שימו לב שכיוון שאנחנו עוסקים בשפה העברית, בכל מקום שאתם קוראים או כותבים טקסט לקובץ, צריך לפתוח את הקובץ עם קידוד של UTF-8.

בכל שורה קראו את תוכן השדה המכיל את שם הקובץ, ואת תוכן השדה שנקרא sentence, שמכיל את המשפט שהדובר מקריא – זהו טקסט הייחוס שלנו (או reference באנגלית). בצעו תמלול של קובץ השמע המתאים, ואת התוצאה שתקבלו רשמו לקובץ התוצאות.

לנוחיותכם, מומלץ שקובץ התוצאות יהיה בעצמו קובץ tsv בעל המבנה הבא (שם הקובץ צריך להיות נקי, ללא שם תיקייה וללא הסיומת של קובץ האודיו):

Filename	Reference Text	Transcribed Text
<filename #1>	*<reference #1>*	*<transcription #1>*
<filename #2>	*<reference #2>*	*<transcription #2>*
...	*...*	*...*
<filename #N>	*<reference #N>*	*<transcription #N>*

זהו הפלט שאתם צריכים להגיש בחלק א'.

חלק ב׳ – תשתית לחישוב דיוק התמלול

לאחר ביצוע התמלול, כתבו סקריפט המשתמש בקובץ התוצאות שייצרתם בשלב הקודם כדי לחשב את **מדדי דיוק התמלול** (ראו להלן): עבור כל קובץ בנפרד, ובאופן כולל עבור הבנצ׳מארק כולו. הפלט שלכם בשלב זה צריך להיות קובץ tsv בעל המבנה הבא (השטח האפור מייצג את הנתונים שתצטרכו לחשב ולמלא):

Filename	N_gt	N_asr	#M	#S	#I	#D	WER	Recall	Precision	F1-Score
<filename #1>										
<filename #2>										
…										
<filename #N>										
TOTAL										

כאשר את ההגדרות למונים N_gt, N_asr, #M, #I, #S, #D וכן למדדי הדיוק WER, Recall, Precision, F1-Score תוכלו למצוא בשיעור מס׳ 4.

זהו הפלט שאתם צריכים להגיש בחלק ב׳ (בנוסף לקוד שיוסבר להלן).

מומלץ לעשות שימוש בפונקציה `align_sequences()` שקיבלתם בתרגיל מספר 2, וכן במחלקה `WordEditWeights` שנתבקשתם לכתוב באותו תרגיל. כפי שראיתם, השימוש במחלקה זו ״מכוון״ את האלגוריתם של יישור המחרוזות להתאים זו מול זו מילים דומות ככל האפשר, במקרה שיש טעויות תמלול ואין התאמה מלאה.

בנוסף, עליכם לממש מחלקה בשם `AccuracyStatistics` שתעזור לכם לחשב את איכות התמלול: של כל קובץ בלבד, ושל הבנצ׳מארק כולו. המחלקה הזו צריכה לאגור את כל הנתונים ולספק את הפונקציות לשם חישוב **מדדי דיוק התמלול** (כלומר, סביר שתתמוך בפונקציות כגון `matches()` או `recall()`):

- שימו לב שכאשר אתם משתמשים בפונקציה `align_sequences()`, היא מחזירה לכם זוגות של מילים מותאמות זו לזו: תוכלו להשתמש בפלט זה על-מנת ליצור את אובייקט ה-`AccuracyStatistics` עבור קובץ בודד, ולקבל את מדדי דיוק התמלול מתוכו.
- בנוסף, המחלקה אמורה לתמוך באופרטור =+ שמאפשר לצבור את הסטטיסטיקות שחושבו על-ידי אובייקט AccuracyStatistics אחר – באופן זה תוכלו לחשב את מדדי דיוק התמלול עבור הבנצ׳מארק כולו.
- שירות נוסף שהמחלקה הזו צריכה לספק הוא צבירת ״טעויות נפוצות״: כלומר בכל פעם שנתקלים בצמד מילים לא מתאימות מתוך הפלט של `align_sequences()` נחשיב אותו כטעות (למען ההבהרה, גם insertion או deletion ייחשבו כטעויות: נוכל לייצג אותם כזוג הכולל מילה ״רגילה״ ומחרוזה ריקה).
אנו נתעניין בטעויות שחוזרות בבנצ׳מארק כולו מספר רב של פעמים. לשם כך, המחלקה שלנו צריכה לתמוך בפונקציה `frequent_errors()`, שמחזירה לנו את הטעויות הנפוצות, בסדר יורד של מספר המופעים שלהם. כלומר, נוכל לכתוב את הקוד הבא (מובא כאן רק לצורך הבהרת ה- interface):

```
for word_pair, num in accuracy_stats.frequent_errors():
    print('-> "%s" replaced by "%s" %d times.' %
          (word_pair[0], word_pair[1], num_occurrences))
```

חלק ג׳ – ניתוח תוצאות ושיפור המדידה

כפי שתראו, מדדי הדיוק שתקבלו בחלק ב׳ אינם טובים במיוחד: אבל כפי שעוד ניווכח, חלק גדול מן הביצועים האלה לא נגרם מטעויות תמלול של המנוע, אלא ניתן לזקוף לחובת בעיות בשיטת המדידה. בחלק זה של התרגיל עליכם לבצע ניתוח של התוצאות ולשפר את דרך המדידה על-מנת לקבל תמונה טובה יותר של איכות התמלול.

המלצות לאיתור בעיות:

- מומלץ לכתוב קובץ לוג מפורט, שבו תדפיסו את היישור (alignment) שקיבלתם עבור כל קובץ. בדקו מה קורה בקבצים שבהם יש WER נמוך (וציון F1 גבוה) ולעומת זאת, מה קורה בקבצים בהם ה-WER גבוה (וציון F1 נמוך).
- הדפיסו לקובץ דו״ח של הטעויות הנפוצות, ונסו לאתר ״טעויות שאינן טעויות״, שנובעות מהבדלי כתיב בין ה-reference ובין התמלול. לצורך העניין הצמד **שנייה/שניה** הוא ״טעות שאינה טעות״ (רק לשם הבהרה, הצורה הראשונה היא הנכונה יותר), אבל הצמד **שמיים/שמים** לא יכול להיחשב כ״טעות שאינה טעות״ כיוון שהמילה השנייה יכולה להיקרא גם שָׁמִים.

כדי לשפר את איכות המדידה, עליכם לכתוב פונקציה בשם `normalize()` שמקבלת טקסט (ואם תרצו, גם פרמטרים נוספים) ומחזירה טקסט מנורמל. שימו לב שמותר לשנות את הטקסט, כולל השמטה או החלפה של תווים מסוימים, אבל לא לשנות את משמעות המילים שבו: לדוגמה, שכמובן לא רלוונטית בעברית, פונקציה של `normalize()` באנגלית יכולה לכלול העברה של הטקסט ל-lower case.

כעת מדדו את איכות התמלול תוך שימוש בטקסטים המקוריים (כלומר בקובץ מחלק א׳), לאחר שאתם מעבירים אותם נרמול. מותר גם לממש שתי פונקציות נפרדות, קרי `normalize_reference()` ו- `normalize_transcription()` אם תגיעו למסקנה שיש צורך בכך, אבל נסו לשתף כמה שיותר קוד בין הפונקציות.

מומלץ בחלק הזה לעבוד באופן איטרטיבי, כלומר:

1. בצעו ניתוח של הבעיות.
2. הכניסו את ההתאמות המתאימות בפונקציות הנרמול.
3. הריצו שוב את מדידת איכות התמלול, והוציאו דו״ח של הטעויות הנפוצות.
4. נסו להגיע ל-WER חד-ספרתי. כל עוד לא הגעתם ו/או שיש לכם רעיונות נוספים לשיפור המדידה, חזרו לשלב 1.

הפלט שאתם צריכים להגיש בחלק ג׳, בנוסף כמובן לקוד, הוא דו״ח דיוק התמלול לאחר ביצוע כל הנרמולים (בדומה לחלק ב׳). כמו-כן עליכם להגיש מסמך שמתאר את השלבים שביצעתם לצורך ניתוח הטעויות, תהליכי הנרמול שביצעתם, ומה היו נתוני הדיוק בסוף כל שלב (לא כל קובץ בנפרד, רק שורה תחתונה עבור הבנצ׳מארק כולו).

חלק ד' – אוגמנטציה של רעשי רקע

בחלק זה נבחן את ההשפעה של תוספת רעשי רקע על איכות התמלול. לשם כך, הורידו את בסיס הנתונים שנקרא Musan מן הקישור הבא:

https://www.openslr.org/17/

שימו לב שלאחר שתפתחו את בסיס הנתונים, תגלו שיש שם תיקייה של קבצי הקלטה של רעשים, ותיקייה נפרדת של מוזיקה (יש גם תיקייה של קבצי דיבור, אבל לא נעשה בה שימוש).

כדי לראות מהם מאפייני הרעש שתעבדו איתם, עליכם לחבר את ספרות הביקורת של מגישי התרגיל מודולו 6. לפי התוצאה שקיבלתם, עבדו עם הנתונים הבאים:

תוצאה	סוג סיגנל רקע	עוצמת סיגנל הרקע	טווח SNR
0	רעש	חזק	0-6 dB
1	מוזיקה	חזק	3-9 dB
2	רעש	בינוני	6-12 dB
3	מוזיקה	בינוני	9-15 dB
4	רעש	חלש	12-18 dB
5	מוזיקה	חלש	15-21 dB

במקרה של קבצי הרעש: אנו מעוניינים להשתמש רק בהקלטות ארוכות יחסית של סביבות רועשות (רעש של מיזוג אוויר, תחנת רכבת וכד'), ולא ברעשים רגעיים כמו טריקת דלת. לכן, עשו שימוש רק בקבצים מתוך התיקייה `noise/free_sound/` שאורכם מעל 30 שניות.

תחילה, יצרו עותק מורעש של קבצי הבנצ'מארק, כאשר עבור כל קובץ מתוך ה-test של CommonVoice בצעו את התהליך הבא:

1. קראו לזיכרון את קובץ ההקלטה; מומלץ לעבוד עם ספריית `soundfile` שיודעת לתמוך בפורמט mp3 (בשאר המקרים תטפלו בקבצים בפורמט wav, שגם בהם הספרייה תומכת).
2. כיוון שהקבצים של Musan הם בתדר דגימה נמוך יותר מאלה של CommonVoice (של 16 קילוהרץ, לעומת קבצי ה-mp3 , שהם בתדר דגימה של 32 קילוהרץ), בצעו הורדת תדר הדגימה של האודיו בהתאם: מומלץ להשתמש לצורך כך בפונקציה `scipy.signal.decimate()` עם פקטור דסימציה של 2. (הורדת קצב הדגימה אינה משפיעה על הביצועים של Whisper, שכן המודל שלו מאומן בכל מקרה על אודיו בתדר דגימה של 16 קילוהרץ.)
3. בחרו באופן רנדומלי קובץ מתוך תיקיית Musan הרלוונטית (רעש או מוזיקה).
4. קראו לזיכרון את קובץ הרקע (קובץ wav), והגרילו נקודת התחלה בו, שתאפשר עדיין חפיפה מלאה עם קובץ הדיבור: כלומר קחו ממנו slice באורך של קובץ הדיבור.

5. הגרילו ערך של SNR בטווח הרלוונטי לכם, והוסיפו את רעש הרקע לסיגנל הדיבור באופן שיתקבל ה-SNR שבחרתם: מדדו את העוצמה של קובץ הדיבור ושל סגמנט הרקע, ומצאו פקטור α בו אתם צריכים להכפיל את הדגימות של סגמנט הרקע לפני שתחברו אותו לסיגנל הדיבור (ראו הדרכה בהרצאה של שיעור מס' 13).
6. רשמו את התוצאה לקובץ שמע: אין צורך לחזור לפורמט של mp3, אפשר ורצוי לשמוע קובץ wav. השאירו את שם הקובץ המורעש כך שיהיה זהה לקובץ המקורי, אבל אחסנו אותו בתיקייה נפרדת.

כדי לתעד את העבודה, רשמו קובץ לוג במבנה ה-tsv הבא:

Filename	**Background file**	**Start point (in seconds)**	**SNR**
<filename #1>	*<background #1>*	*<start #1>*	*<SNR #1>*
<filename #2>	*< background #2>*	*<start #2>*	*<SNR #2>*
…	*…*	*…*	*…*
<filename #N>	*< background #N>*	*<start #N>*	*<SNR #N>*

כעת, עליכם לתמלל את כל הקבצים בתיקייה ה"מורעשת" וליצור קובץ תמלול, כפי שעשיתם בחלק א'. על הקובץ הזה בצעו חישוב מדדי איכות, כמובן תוך שימוש בפונקציות הנרמול שכתבתם בחלק ג'. אל תשנו את פונקציות הנרמול ואת אופן המדידה בשלב הזה. המטרה היא כמובן לראות עד כמה הרעש פוגע באיכות התמלול, ובאילו מקרים.

הפלט שאתם צריכים להגיש, בנוסף לסקריפט ש"מרעיש" את בסיס הנתונים, הוא קובץ הלוג שמתואר לעיל, קובץ התמלול ודו"ח איכות התמלול שקיבלתם על החומר המורעש.

בהצלחה!